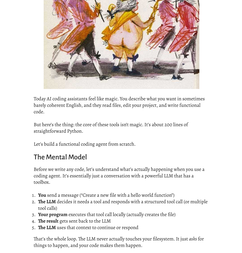

The Emperor Has No Clothes: How to Code Claude Code in 200 Lines of Code

January 2025

Today AI coding assistants feel like magic. You describe what you want in sometimes barely coherent English, and they read files, edit your project, and write functional code.

But here's the thing: the core of these tools isn't magic. It's about 200 lines of straightforward Python.

Let's build a functional coding agent from scratch.

The Mental Model

Before we write any code, let's understand what's actually happening when you use a coding agent. It's essentially just a conversation with a powerful LLM that has a toolbox.

1. **You** send a message ("Create a new file with a hello world function")
2. **The LLM** decides it needs a tool and responds with a structured tool call (or multiple tool calls)
3. **Your program** executes that tool call locally (actually creates the file)
4. **The result** gets sent back to the LLM
5. **The LLM** uses that context to continue or respond

That's the whole loop. The LLM never actually touches your filesystem. It just *asks* for things to happen, and your code makes them happen.

Three Tools You Need

Our coding agent fundamentally needs three capabilities:

- **Read files** so the LLM can see your code
- **List files** so it can navigate your project
- **Edit files** so it can give the directive to create and modify code

That's it. Production agents like Claude Code have a few more capabilities including `grep`, `bash`, `websearch`, etc but for our purposes we'll see that three tools is sufficient to do incredible things.

Setting Up the Scaffolding

We start with basic imports and an API client. I'm using OpenAI here, but this works with any LLM provider:

```
import inspect
import json
import os

import anthropic
from dotenv import load_dotenv
from pathlib import Path
from typing import Any, Dict, List, Tuple

load_dotenv()

claude_client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"
```

Some terminal colors to make outputs readable:

```
YOU_COLOR = "\u001b[94m"
ASSISTANT_COLOR = "\u001b[93m"
RESET_COLOR = "\u001b[0m"
```

And a utility to resolve file paths (so `file.py` becomes `/Users/you/project/file.py`):

```
def resolve_abs_path(path_str: str) -> Path:
    """
    file.py -> /Users/you/project/file.py
    """
    path = Path(path_str).expanduser()
    if not path.is_absolute():
        path = (Path.cwd() / path).resolve()
    return path
```

Implementing the Tools

Note you should be detailed about your tool function docstrings as they will be used by the LLM to reason about what tools should be called during the conversation. More on this below.

Tool 1: Read File

The simplest tool. Take a filename, return its contents:

```
def read_file_tool(filename: str) -> Dict[str, Any]:
    """
    Gets the full content of a file provided by the user.
    :param filename: The name of the file to read.
    :return: The full content of the file.
    """
    full_path = resolve_abs_path(filename)
    print(full_path)
    with open(str(full_path), "r") as f:
        content = f.read()
    return {
        "file_path": str(full_path),
        "content": content
    }
```

We return a dictionary because the LLM needs structured context about what happened.

Tool 2: List Files

Navigate directories by listing their contents:

```
def list_files_tool(path: str) -> Dict[str, Any]:
    """
    Lists the files in a directory provided by the user.
    :param path: The path to a directory to list files from.
    :return: A list of files in the directory.
    """
    full_path = resolve_abs_path(path)
    all_files = []
    for item in full_path.iterdir():
        all_files.append({
            "filename": item.name,
            "type": "file" if item.is_file() else "dir"
        })
    return {
        "path": str(full_path),
        "files": all_files
    }
```

Tool 3: Edit File

This is the most complex tool, but still straightforward. It handles two cases:

- **Creating a new file** when `old_str` is empty
- **Replacing text** by finding `old_str` and replacing with `new_str`

```
def edit_file_tool(path: str, old_str: str, new_str: str) -> Dict[str, Any
    """
    Replaces first occurrence of old_str with new_str in file. If old_str
    create/overwrite file with new_str.
    :param path: The path to the file to edit.
    :param old_str: The string to replace.
    :param new_str: The string to replace with.
    :return: A dictionary with the path to the file and the action taken.
    """
    full_path = resolve_abs_path(path)
    if old_str == "":
        full_path.write_text(new_str, encoding="utf-8")
        return {
            "path": str(full_path),
            "action": "created_file"
        }
    original = full_path.read_text(encoding="utf-8")
    if original.find(old_str) == -1:
        return {
            "path": str(full_path),
            "action": "old_str not found"
        }
    edited = original.replace(old_str, new_str, 1)
    full_path.write_text(edited, encoding="utf-8")
    return {
        "path": str(full_path),
        "action": "edited"
    }
```

The convention here: empty `old_str` means "create this file." Otherwise, find and replace. Real IDEs add sophisticated fallback behavior when the string isn't found, but this works.

The Tool Registry

We need a way to look up tools by name:

```
TOOL_REGISTRY = {
    "read_file": read_file_tool,
    "list_files": list_files_tool,
    "edit_file": edit_file_tool
}
```

Teaching the LLM About Our Tools

The LLM needs to know what tools exist and how to call them. We generate this dynamically from our function signatures and docstrings:

```
def get_tool_str_representation(tool_name: str) -> str:
    tool = TOOL_REGISTRY[tool_name]
    return f"""
    Name: {tool_name}
    Description: {tool.__doc__}
    Signature: {inspect.signature(tool)}
    """

def get_full_system_prompt():
    tool_str_repr = ""
    for tool_name in TOOL_REGISTRY:
        tool_str_repr += "TOOL\n===" + get_tool_str_representation(tool_n
        tool_str_repr += f"\n{'='*15}\n"
    return SYSTEM_PROMPT.format(tool_list_repr=tool_str_repr)
```

And the system prompt itself:

```
SYSTEM_PROMPT = """
You are a coding assistant whose goal it is to help us solve coding tasks.
You have access to a series of tools you can execute. Here are the tools y

{tool_list_repr}

When you want to use a tool, reply with exactly one line in the format: 't
Use compact single-line JSON with double quotes. After receiving a tool_re
If no tool is needed, respond normally.
"""
```

This is the key insight: we're just telling the LLM "here are your tools, here's the format to call them." The LLM figures out when and how to use them.

Parsing Tool Calls

When the LLM responds, we need to detect if it's asking us to run a tool:

```
def extract_tool_invocations(text: str) -> List[Tuple[str, Dict[str, Any]]
    """
    Return list of (tool_name, args) requested in 'tool: name({...})' line
    The parser expects single-line, compact JSON in parentheses.
    """
    invocations = []
    for raw_line in text.splitlines():
        line = raw_line.strip()
        if not line.startswith("tool:"):
            continue
        try:
            after = line[len("tool:"):].strip()
            name, rest = after.split("(", 1)
            name = name.strip()
            if not rest.endswith(")"):
                continue
            json_str = rest[:-1].strip()
            args = json.loads(json_str)
            invocations.append((name, args))
        except Exception:
            continue
    return invocations
```

Simple text parsing. Look for lines starting with `tool:`, extract the function name and JSON arguments.

The LLM Call

A thin wrapper around the API:

```
def execute_llm_call(conversation: List[Dict[str, str]]):
    system_content = ""
    messages = []

    for msg in conversation:
        if msg["role"] == "system":
            system_content = msg["content"]
        else:
            messages.append(msg)

    response = claude_client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=2000,
        system=system_content,
        messages=messages
    )
    return response.content[0].text
```

The Agent Loop

Now we put it all together. This is where the "magic" happens:

```
def run_coding_agent_loop():
    print(get_full_system_prompt())
    conversation = [{
        "role": "system",
        "content": get_full_system_prompt()
    }]
    while True:
        try:
            user_input = input(f"{YOU_COLOR}You:{RESET_COLOR}:")
        except (KeyboardInterrupt, EOFError):
            break
        conversation.append({
            "role": "user",
            "content": user_input.strip()
        })
        while True:
            assistant_response = execute_llm_call(conversation)
            tool_invocations = extract_tool_invocations(assistant_response
            if not tool_invocations:
                print(f"{ASSISTANT_COLOR}Assistant:{RESET_COLOR}: {assista
                conversation.append({
                    "role": "assistant",
                    "content": assistant_response
                })
                break
            for name, args in tool_invocations:
                tool = TOOL_REGISTRY[name]
                resp = ""
                print(name, args)
                if name == "read_file":
                    resp = tool(args.get("filename", "."))
                elif name == "list_files":
                    resp = tool(args.get("path", "."))
                elif name == "edit_file":
                    resp = tool(args.get("path", "."),
                                args.get("old_str", ""),
                                args.get("new_str", ""))
                conversation.append({
                    "role": "user",
                    "content": f"tool_result({json.dumps(resp)})"
                })
```

The structure:

1. **Outer loop**: Get user input, add to conversation
2. **Inner loop**: Call LLM, check for tool invocations
    - If no tools needed, print response and break inner loop
    - If tools needed, execute them, add results to conversation, loop again

The inner loop continues until the LLM responds without requesting any tools. This lets the agent chain multiple tool calls (read a file, then edit it, then confirm the edit).

Running It

```
if __name__ == "__main__":
    run_coding_agent_loop()
```

Now you can have conversations like:

> ***You:** Make me a new file called hello.py and implement hello world in it*

Agent calls *edit_file* with path="hello.py", old_str="", new_str="print('Hello World')"

> ***Assistant:** Done! Created hello.py with a hello world implementation.*

Or multi-step interactions:

> ***You:** Edit hello.py and add a function for multiplying two numbers*

Agent calls *read_file* to see current contents. Agent calls *edit_file* to add the function.

> ***Assistant:** Added a multiply function to hello.py.*

What We Built vs. Production Tools

This is about 200 lines. Production tools like Claude Code add:

- **Better error handling** and fallback behaviors
- **Streaming responses** for better UX
- **Smarter context management** (summarizing long files, etc.)
- **More tools** (run commands, search codebase, etc.)
- **Approval workflows** for destructive operations

But the core loop? It's exactly what we built here. The LLM decides what to do, your code executes it, results flow back. That's the whole architecture.

Try It Yourself

The full source is about 200 lines. Swap in your preferred LLM provider, adjust the system prompt, add more tools as an exercise. You'll be surprised how capable this simple pattern is.

If you're interested in learning state-of-the-art AI software development techniques for professional engineers, check out my online course.

PYTHON AI LLM CODING-AGENTS TUTORIAL

[redacted]

Like what you read? I would love to hear from you! 🙂

X Post X Follow